

Relatório de Estágio : Avaliação de Plataformas de Gestão de Dispositivos ESP32

Francisco José Nogueira da Rocha Matos

7 de agosto de 2026

Conteúdo

1	Introdução	2
2	Zephyr RTOS	2
3	ESP-IDF Nativo	2
4	ThingsBoard	3
5	ESP RainMaker	3
6	Azure IoT Hub + Device Provisioning Service (DPS)	4
7	Síntese Comparativa	4
7.1	OTA (dispositivo)	4
7.2	Plataformas de Gestão	4
7.3	Ver vs. Gerir: Quem Implementa	5
7.4	Drivers I2C e LoRa: Zephyr vs. ESP-IDF	5
7.5	Prova de conceito: driver RN2483 (LoRa)	5
8	Mongoose OS	5
9	Conclusão	6

1 Introdução

Durante este estágio de 1 mês na Watgrid, avalei plataformas e sistemas operativos para o ESP32, com o objetivo de identificar quais melhor suportam o caso de uso real da empresa. O foco inicial foi a atualização remota de firmware (OTA), alargado depois à facilidade de integração de *drivers* de sensores I2C e comunicação LoRa. Foram avaliados o Zephyr RTOS e o ESP-IDF nativo (camada de firmware), o ESP RainMaker, o ThingsBoard e o Azure IoT Hub + DPS (plataformas de gestão de dispositivos na *cloud*), e ainda o Mongoose OS como candidato adicional. .

2 Zephyr RTOS

- Conectividade Wi-Fi validada, com ligação automática programática.
- OTA via MCUboot+mcumgr (série): funcional, modo *upgrade-only* (sem *rollback*).
- OTA via LwM2M/Leshan (Wi-Fi): validado com hardware real, sem falhas – 680KB, *rollback* do MCUboot aplicado com sucesso. Requer um *patch* adicional ao checkout (*issue* #77952/PR #78121), ainda não incluído numa *release* oficial.
- *Drivers* de sensores I2C e LoRa: subsistema nativo *in-tree* (**drivers/sensor/**, **drivers/lora/**). IIS2ICLX com *driver* nativo (I2C+SPI); KX122 e MCP3221 não encontrados; LoRa nativo e completo (SX126x/SX127x/RYLR).

3 ESP-IDF Nativo

- OTA via HTTP **sem qualquer falha**, com Wi-Fi ativo durante a escrita – exatamente o cenário que falhava no Zephyr, confirmando que a causa é do *port* Zephyr, não do hardware/rede.
- *Rollback* automático confirmado: imagem defeituosa (**abort()**) reverte sozinha para a versão estável, com proteção contra repetição automática.
- Esforço de integração baixo, sem as dificuldades de Kconfig/*devicetree* do Zephyr.
- *Drivers* de sensores I2C e LoRa: sem subsistema nativo (só periféricos genéricos – I2C/SPI/ADC interno). IIS2ICLX, KX122 e MCP3221 ausentes do ESP Component Registry; LoRa sem suporte nativo, mas com componentes de terceiros maduros (**radiolib**, **sx127x**).

4 ThingsBoard

- Ligação MQTT com *access token*; telemetria, *shared attributes* (config remota) e RPC (*getStatus/reboot*) validados.
- OTA próprio sobre MQTT (blocos com *checksum* incremental) – sem falhas.
- *Rollback* confirmado; **achado**: proteção anti-repetição não vinha incluída, causando ciclo infinito download/falha/reversão – corrigido com memória de falha em NVS.
- Provisionamento automático (credencial genérica → token individual) e alarme de desconexão com email, ambos validados.
- Lacuna de segurança identificada e corrigida: transporte migrado de `mqtt://` para `mqtt://`.
- Esforço de integração baixo, sem cartão de crédito/débito, grátis até 5 dispositivos.
- **PoC Zephyr + ThingsBoard**: ao contrário do RainMaker (SDK preso ao ESP-IDF), o ThingsBoard só precisa de um cliente MQTT – escrito e validado com hardware real sobre o MQTT nativo do Zephyr (provisionamento, MQTTS, telemetria).

5 ESP RainMaker

- Esforço de integração inicial **mais baixo de todas as plataformas** (*zero-config*, gestor de dependências automático).
- **Nota – *vendor lock-in***: SDK proprietário, só funciona em ESP32/Espressif – ao contrário do ThingsBoard/Azure (qualquer fabricante). Esforço mais baixo tem esta contrapartida.
- Provisionamento via BLE + app móvel obrigatória.
- OTA e *rollback* ativos por omissão, sem código adicional.
- RPC customizado validado; *Node Groups* (gestão de frota) confirmados só a nível conceptual (um único dispositivo).
- Notificação de desconexão *push*, sem qualquer configuração prévia.
- **Reavaliação**: Reboot e Wi-Fi Reset nativos (*System Service*, zero código, testados com sucesso via CLI/console/app); RSSI e saúde de memória nativos via *Insights* (*dashboard* incluído, sem código nosso); versão de firmware exposta como metadado nativo. Resultado: **única das três plataformas com as 7 capacidades testadas nativas** (ThingsBoard e Azure só têm 1 cada).
- **Fragilidade do provisionamento**: reprovisionar a mesma placa (reutilizada de outros testes) falhou repetidamente via BLE/app oficial (*ESPSessionError*), só resolvido com limpeza completa da flash – e voltou a falhar numa tentativa seguinte.

6 Azure IoT Hub + Device Provisioning Service (DPS)

Avaliação prática subsequente (exige cartão de crédito/débito).

- **Provisionamento:** DPS + IoT Hub (*tier* Free) configurados; SAS tokens calculados localmente (HMAC-SHA256), exigindo sincronização SNTP. Testado com sucesso tanto *Individual* como **Group Enrollment** (chave partilhada, cada dispositivo deriva a sua própria chave – duas identidades distintas coexistentes confirmadas).
- **Direct Methods** (RPC) e **Device Twin** (config remota *desired/reported*) validados com sucesso.
- **OTA:** serviço gerido oficial indisponível no *tier* Free – sinalizado via Device Twin, reproveitando o mecanismo HTTP do ESP-IDF nativo. Sucesso confirmado; teste de falha revelou e permitiu corrigir um **bug real** na proteção anti-ciclo (ordem de reconciliação pós-*reboot*), com paralelo direto ao mesmo problema já encontrado no ThingsBoard.
- **Gestão de frota:** *Configurations + Deployments* validado com dois dispositivos (um real, um simulado), superando o nível apenas conceptual do RainMaker.
- **Alarmes:** configuração explorada até ao fim, mas não ativada – único recurso do PoC com custo mensal explícito (\$0.10 USD), decisão deliberada de âmbito.
- **C2D messages** (mensagem pontual assíncrona, cloud→dispositivo) e **Message Routing** (filtragem de mensagens por condição) validados – exploração lateral, mais sobre dados do que sobre gestão do dispositivo.
- Esforço de integração no firmware alto (protocolo do DPS e SAS tokens implementados à mão, sem SDK oficial pesado).

7 Síntese Comparativa

7.1 OTA (dispositivo)

Critério	MCUboot	LwM2M	ESP-IDF	ThingsBoard	RainMaker
Transporte	Série	Wi-Fi/CoAP	Wi-Fi/HTTP	Wi-Fi/MQTT	Wi-Fi/HTTPS
Concluiu	Sim	Não	Sim	Sim	Sim
<i>Rollback</i>	Não	Não testado	Confirmado	Confirmado	Por omissão
Esforço	Alto	Alto	Baixo	Baixo	Muito baixo

7.2 Plataformas de Gestão

Critério	ESP-IDF	ThingsBoard	RainMaker	Azure DPS+Hub
Hardware suportado	Só ESP32	Qualquer	Só ESP32/Espressif	Qualquer
Dashboards	Não	Sim	Sim	Não testado
Config. remota	Não	Sim	Sim	Sim, confirmado
RPC	Não	Sim	Sim	Sim, confirmado
OTA via cloud	Não	Sim	Sim	Parcial
<i>Rollback</i>	Confirmado	Confirmado	Confirmado	Confirmado (bug corrigido)
Alarmes	Não	Sim	Sim (espontâneo)	Explorado, não ativado (custo)
Provisionamento	Não	Sim	Sim (app/BLE)	Sim (individual + grupo)
Gestão de frota	Não	Sim	Só conceptual	Sim, confirmado
Esforço inicial	Baixo	Baixo	Muito baixo	Alto
Custo	Grátis	Grátis (5 disp.)	Grátis (20 disp.)	Grátis (<i>tier</i> Free)

7.3 Ver vs. Gerir: Quem Implementa

Distinção relevante: separar **ver** (leitura de estado) de **agir** (comando), e se cada um vem pronto da **plataforma** ou foi feito **manualmente**.

Capacidade	ThingsBoard	RainMaker	Azure (DPS+Hub)
Ver versão de firmware	Manualmente (telemetria)	Plataforma (metadado nativo do nó)	Manualmente (<i>reported</i>)
Disparar atualização (OTA)	Híbrido (pacote: plataforma; download: manual)	Plataforma (console própria, zero código)	Manualmente (Device Twin)
Reiniciar remotamente	Manualmente (RPC customizado)	Plataforma (<i>System Service</i> nativo)	Manualmente
Ver força de sinal Wi-Fi (RSSI)	Não testado	Plataforma (Insights, zero código)	Manualmente (<i>wifiRssi</i> no <i>twin</i>)
Ver estado de conectividade	Plataforma (<i>active</i> , nativo)	Plataforma (notificação espontânea)	Plataforma (<i>connectionState</i>)
Agir sobre conectividade	Não testado	Plataforma (<i>Wi-Fi Reset</i> nativo)	Não testado
Ver memória/saúde geral	Manualmente (RPC customizado)	Plataforma (Insights, zero código)	Manualmente (<i>twin</i> + Direct Method)

RainMaker: único com todas as 7 capacidades nativas (0 código nosso); ThingsBoard e Azure têm só 1 cada (conectividade).

7.4 Drivers I2C e LoRa: Zephyr vs. ESP-IDF

Critério	Zephyr	ESP-IDF
Subsistema nativo de <i>drivers</i> de sensores	Sim	Não
IIS2ICLX	Nativo	Não encontrado
KX122 / MCP3221	Não encontrados	Não encontrados
LoRa nativo	Sim (SX126x/SX127x/RYLR)	Não (terceiros: radiolib, sx127x)

Nenhum *framework* tem os 3 sensores prontos, mas o Zephyr trata sensores/LoRa como parte do próprio SO (API unificada); no ESP-IDF depende sempre de bibliotecas de terceiros.

Nota sobre módulos LoRa concretos (RN2483, WLR089U0): são módulos com comando por texto via UART, não transceptores SPI *nus* – nenhum *framework* tem isto pronto.

7.5 Prova de conceito: driver RN2483 (LoRa)

Escrito e compilado com sucesso um segundo driver *out-of-tree* para Zephyr, agora para o módulo RN2483 (comandos de texto via UART, não SPI). Liga-se à mesma API nativa de LoRa (*lora_config/lora_send*) – do ponto de vista de quem usa, é intercambiável com um transceptor nativo. 302 linhas totais (mais que o MCP3221, por ter mais comandos de configuração de rádio), compilou à primeira, sem *warnings*. Sem módulo físico, validado só ao nível de compilação/API, não de comunicação real.

8 Mongoose OS

Testado com o mesmo rigor prático (instalação, compilação, gravação real):

- **Manutenção:** último *release* dez/2021, último código real fev/2023 (correção ESP32-C3) – commits recentes são só links no README.
- **O que o substituiu:** a Cesanta não desapareceu – passou a concentrar-se na *Mongoose Library* (*cesanta/mongoose*), uma biblioteca de rede em C (TCP/IP, HTTP, MQTT, TLS, agora com *rollback* de OTA), usada **dentro** de outro sistema (bare-metal/FreeRTOS/Zephyr/ESP-IDF), não um SO próprio. Muito ativa: *releases* a cada poucos meses e código novo até à véspera desta consulta. Âmbito bem mais estreito que o Mongoose OS (só rede, sem drivers/OTA de app/scripting) – não avaliada em profundidade, fica como referência.
- **Teste prático:** instalação, compilação (*demo-c*) e gravação bem-sucedidas no hardware real – **ainda funciona hoje**. Achado: *mos build* compila na nuvem da Cesanta, não localmente (dependência extra, ponto único de falha, ao contrário de Zephyr/ESP-IDF).
- **Drivers:** mesmo padrão de Zephyr/ESP-IDF – KX122, IIS2ICLX e MCP3221 não têm biblioteca pronta; sem biblioteca de LoRa.

- **Features notáveis (não adotado, mas de registrar):** *scripting* JavaScript sem recompilar (mJS), dashboard/OTA próprio (mDash), config remota nativa, RPC extensível (RPC.List), provisionamento pronto para AWS/Azure/GCP/Watson, hardware suportado (ESP32, ESP8266, STM32, CC3200/3220) – nenhum framework avaliado tem equivalente.

Conclusão: não está avariado, só sem evolução e dependente de infraestrutura de nuvem de terceiros – risco de manutenção, não limitação técnica.

9 Conclusão

O **RainMaker** é quem melhor cumpre o critério – único com as 7 capacidades de gestão do dispositivo nativas, zero código nosso. **Mas** tem duas reservas: (1) o provisionamento via BLE/app mostrou-se pouco fiável na prática (falhas de sessão repetidas, só resolvidas com limpeza total da flash) – risco operacional real para uma implantação com vários sensores em campo, sem garantia de técnico com telemóvel disponível sempre que preciso; (2) é um SDK proprietário, só ESP32/Esspressif – compromete a Watgrid a este fornecedor de hardware, sem a flexibilidade de diversificar chips que o ThingsBoard/Azure permitem.

O **ThingsBoard** continua a opção mais equilibrada se a fiabilidade/simplicidade do provisionamento (sem app/BLE) pesar mais do que a natividade capacidade-a-capacidade – ao custo de implementar manualmente quase tudo. Reserva: limite de 5 dispositivos grátis.

A **Azure** (testada em profundidade nesta ronda) iguala ou supera o ThingsBoard tecnicamente – incluindo *Group Enrollment* e gestão de frota, que superam mesmo o RainMaker – mas com o maior esforço de integração de firmware das três. Melhor candidato para escalar além do limite do ThingsBoard.

Recomendação (gestão do dispositivo): RainMaker como foco imediato (melhor no critério que importa agora), **condicionado** a validar a fiabilidade do provisionamento em escala real antes de comprometer. ESP-IDF + ThingsBoard como alternativa segura e já validada de ponta a ponta caso essa validação falhe.

Firmware: com LoRa confirmado como requisito crítico do projeto, e sendo nativo só no Zephyr (não no ESP-IDF), o Zephyr é candidato firme para a camada de firmware – condicionado a aplicar um *patch* adicional para OTA fiável sobre Wi-Fi, validado com hardware real, e a assumir o custo de *drivers* próprios para KX122/MCP3221.

Decisão a resolver: o RainMaker corre sobre ESP-IDF, não sobre Zephyr – se o LoRa nativo do Zephyr for requisito duro, isso é incompatível com o RainMaker. **Zephyr + ThingsBoard** deixou de ser só teórico: validado com hardware real (provisionamento, MQTTS, telemetria, sem SDK de firmware específico exigido) – é a combinação recomendada nesse cenário.